Dynamic Inter-Cluster Resource Sharing and Node Migration in Hierarchical Federated Fog Computing for IoT Deployments

Narek Naltakyan
National Polytechnic University of Armenia
Yerevan, Armenia
nareknaltakyan1@gmail.com

Abstract*—Hierarchical federated fog computing architectures often have serious difficulties when a cluster is in a heavy load for a long time and it doesn't have enough local capacity for the requests. In this paper a new model of inter-cluster resource sharing is presented which makes it possible to redistribute the load dynamically. It includes two main strategies: (1) request redirection through direct gateway-to-gateway connections, and (2) temporary node migration between clusters. To be in control of the system, the architecture uses intelligent master node coordination, applying a limit of 30% for the maximum resource sharing so none of the clusters becomes unstable. The proposed architecture is implemented using iFogSim2 simulation framework. Results of the experiment shows a reduction for 68% in request dropping, an improvement for 42% in the use of resource, and system-wide availability for 99.2%.

Keywords*—Federated Fog Computing, Inter-Cluster Resource Sharing, Node Migration, Load Balancing, IoT Scalability, iFogSim2*

I. INTRODUCTION

The increase in Internet of Things (IoT) deployments has fog computing architectures developed from basic single-cluster models to advanced systems that stretch across different geographic areas [1]. However, when a single cluster reaches the highest load capacity, the system has no way to make use of available resources in neighboring clusters.

Modern fog computing deployments often have serious problems. For example when isolated resource pools lead to cases where some clusters become heavily overloaded while other nearby clusters have plenty of unused capacity, the imbalance of geographic load creates cases when busy clusters have request drops despite the fact that the system has wide availability, and there are no standardized protocols that allow clusters to share resources safely while keeping the stability of the system. These problems are especially visible in deployments of smart cities where IoT traffic patterns can change for a great deal depending on geographic regions and temporal windows.

A smart city example shows it clearly. Imagine a large event that causes massive IoT traffic in one district while districts that surround are operating at 40% capacity. The busy cluster is forced by the current architectures to drop requests even though nearby clusters have enough power to help. In this paper the limitation is addressed by introducing intelligent inter-cluster resource sharing.

A. Contributions

In this paper several contributions are presented: (1) A resource sharing architecture consisting of two parts that uses both redirection of requests and node migration of nodes to handle the cases of overload. (2) A master node coordination protocol which has safety limit for 30% to stop the cascade failures. (3) A master-of-masters logging system that keeps less than 8ms of overhead. (4) A full iFogSim2 simulation which shows a decrease for 68% in request dropping and an increase for 42% in the use of resources across the system.

B. Paper Organization

The remainder of this paper proceeds as follows. Section II reviews related work on fog computing architectures, load balancing frameworks, and inter-cluster resource management, positioning our contribution relative to existing approaches. Section III describes the system architecture, detailing the Gateway Connection Manager, Master Node Resource Negotiator, and resource sharing modes (request redirection and node migration). Section IV presents the decision algorithms, including load calculation formulas, threshold definitions, and the master node decision pseudocode with priority calculation mechanisms. Section V describes the implementation in iFogSim2, including simulation topology, workload models, custom component development, and solutions to implementation challenges such as migration checkpoints, load prediction, and network partition handling. Section VI presents experimental evaluation results across four scenarios (geographic hotspot, temporal spike, cascading load, and gradual growth), comparing baseline, redirection-only, and full system configurations. Section VII discusses security and trust considerations, comparative analysis with alternative approaches (cloud forwarding and static federation), and provides a detailed smart city deployment case study demonstrating real-world applicability. Section VIII concludes with a summary of contributions and outlines future research directions including machine learning-based load prediction, multi-tier resource sharing, application-aware QoS guarantees, enhanced security mechanisms, and formal verification of decision algorithms.

II. RELATED WORK

Fog computing [5] was developed to solve issues of latency and limited bandwidth with pure cloud computing architectures [1]. The paradigm brings computational resources closer to IoT devices, enabling real-time processing and reducing network congestion. More current surveys [3][4] indicate that resource orchestration and load balancing are among the most pressing issues when working with fog computing [10].

Hierarchical Fog Architectures. Qayyum et al. [6] put forward a mobility-aware hierarchical fog computing model that does allow for some level of dynamic resource allocation as opposed to traditional static approaches. Their work demonstrates the importance of geographical hierarchy in fog deployments but focuses primarily on intra-cluster mobility rather than inter-cluster resource sharing. Similarly, Mahmud et al. [4] provide a comprehensive taxonomy of fog computing architectures, identifying the need for federated approaches but not addressing the specific mechanisms for resource negotiation between autonomous clusters.

Load Balancing Frameworks. Existing load-balancing techniques [6][7] are not very effective when they are applied to fog environments. Singh et al. [7] explore server-storage virtualization techniques that work well in data center environments but fail to account for the heterogeneous nature and geographical constraints of fog nodes. Cardellini et al. [8] present dynamic load balancing algorithms for distributed systems, yet their approaches assume homogeneous resources and centralized control, which contradicts the federated nature of multi-cluster fog architectures. These traditional approaches struggle with the unique challenges of fog computing: resource heterogeneity, geographical distribution, and autonomous cluster management.

Intra-Cluster Resource Management. Much of the recent fog-specific research done so far [8][9] has focused on the amount of data transferred within clusters of fog nodes and not on the resource sharing that takes place between clusters. Xu et al. [9] investigate workflow scheduling in cloud-fog environments, optimizing task placement within a single cluster hierarchy. While their work addresses vertical resource allocation (from cloud to fog to edge), it does not consider horizontal collaboration between peer fog clusters. Puthal et al. [10] examine secure load balancing mechanisms within fog computing but limit their scope to single-cluster scenarios, leaving inter-cluster security protocols unexplored.

Federated and Multi-Cluster Systems. Limited research exists on federated fog computing where multiple autonomous clusters collaborate. Most existing work assumes either complete centralization or complete isolation. The gap between these extremes—where clusters maintain autonomy while selectively sharing resources—remains largely unaddressed. Previous federated approaches typically involve static resource reservation or complete trust assumptions, neither of which is practical for real-world deployments where clusters may belong to different organizations or administrative domains.

Resource Migration and Virtualization. While virtual machine migration has been extensively studied in cloud computing contexts, fog node migration presents unique challenges due to stateful IoT connections, real-time processing requirements, and limited bandwidth between clusters. Existing migration techniques designed for cloud environments introduce unacceptable service interruptions when applied to latency-sensitive fog workloads.

The ideas presented in this paper build upon these foundational principles to develop a comprehensive mechanism for inter-cluster resource sharing that combines request redirection for immediate load relief with node migration for sustained imbalances, while maintaining cluster autonomy and system stability through intelligent coordination protocols.

III. SYSTEM ARCHITECTURE

Our proposed design of inter-cluster resource sharing extends HFMCFA with three new pieces: The system uses a four-layer hierarchy which is shown in Figure 1.

1. Gateway Connection Manager,
2. Master Node Resource Negotiator
3. Master-of-Masters Transaction Logger.

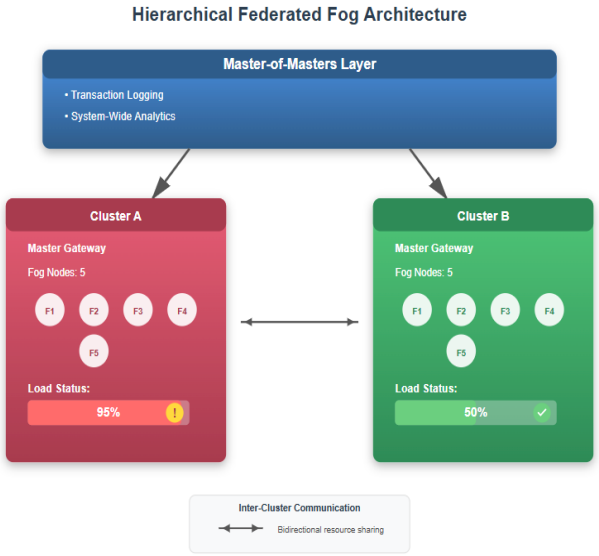

Fig. 1. Hierarchical federated multi-cluster architecture with master-of-masters coordination layer and inter-cluster resource sharing.

A. Gateway Connection Manager

The Gateway Connection Manager makes channels secured between cluster gateways with the use of TLS 1.3 encryption. For secure IoT communication, our gateway manager keeps the pre-established connections active, follows the volume of forwarded requests to make sure the 30% sharing limit is not exceeded, and automatically detects any occurred failure.

B. Master Node Resource Negotiator

The Master Node Resource Negotiator controls all intelligent decision-making. Each master node monitors its own local load (CPU, memory, queue depth). It also calculates the available sharing capacity (0-30%), checks the status of neighbors, processes resource sharing requests, and manages the coordination of node migration between clusters.

C. Resource Sharing Modes

Redirection of requests offers temporary support during sudden load spikes and requires less than 50ms for negotiation. When Cluster A realizes that all of its fog nodes exceed the capacity of 90%, the master node checks which neighboring node still has available resources and sends a request of redirection to it. If the neighbor approves, the gateway of Cluster A starts to send future requests to the neighbor's gateway, which processes them and returns the results. This mode adds latency but is fast to activate.

Node migration is used for prolonged load imbalances when a cluster is overloaded for a sustained time. Even though this process of migration takes 30-60 seconds, it doesn't add additional latency after completing the migration. A designated fog node finishes its current tasks, disconnects from its original cluster, registers with the target cluster, and starts to accept new requests. The node functions as a full member until load becomes normal or the original cluster wants it back.

IV. DECISION ALGORITHMS

The master node decision algorithm dictates the time and the way to share resources. Load is calculated using

$$Load = 0.4 \times CPU + 0.3 \times Queue + 0.3 \times Memory.$$

Algorithm Pseudocode.

Algorithm shows the complete master node decision algorithm. The algorithm performs every monitoring interval (default: 30 seconds) to check the load of cluster and start to share resources when there is a need.

Algorithm: Master Node Resource Sharing Decision
Input: ClusterState S, NeighborList N
Output: ResourceSharingDecision

1:Load ← 0.4×S.CPU + 0.3×S.Queue + 0.3×S.Memory
2: Trend ← (Load - S.PreviousLoad) / MonitorInterval
3: if Load < 0.85 OR Trend ≤ 0 then
4: return NO_ACTION
5: SharingRatio ← S.SharedCapacity / S.TotalCapacity
6: Mode ← (SharingRatio < 0.30) ? REDIRECT : MIGRATE
7: CandidateList ← FilterNeighbors(N, Mode)
8: for each neighbor ∈ CandidateList do
9: Priority ← CalculatePriority(neighbor)
10: BestNeighbor ← argmax(Priority)
11: Response ← SendRequest(BestNeighbor, Mode)
12: if Response = APPROVED then
13: ActivateSharing(BestNeighbor, Mode)
14: LogTransaction(S, BestNeighbor, Mode)

In Figure 2 the complete decision flow is shown. The algorithm uses different thresholds: request assistance when Load exceeds 85%, approve redirection when Load is below 70%, approve migration when Load is below 75%, maximum sharing of 30%, and minimum reciprocity of 0.6. The reciprocity score equals resources_provided divided by resources_received plus one, ensuring fair resource exchange.

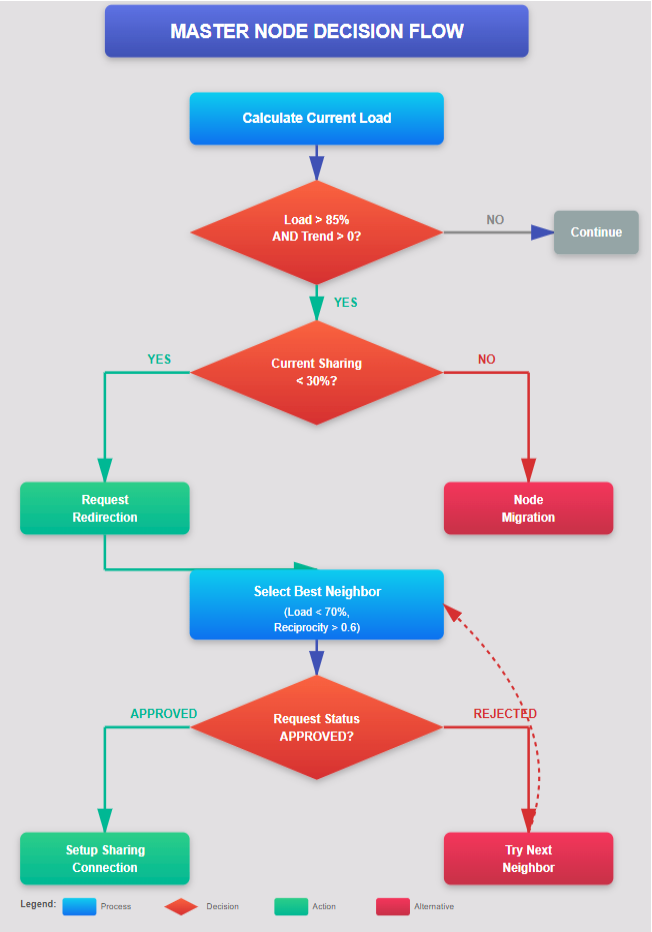

Fig. 2. Master node decision flow showing load evaluation, mode selection, neighbor selection, and transaction logging.

The priority calculation function (line 9) uses a method for weighted scoring. For each possible neighbor, we compute:

$$Priority = 0.4 \times ReciprocityScore + 0.3 \times CriticalityFactor + 0.2 \times AvailableCapacity + 0.1 \times ProximityScore$$

The reciprocity score ranges from 0.0 to 2.0, where 1.0 means that clusters exchange resources in a balanced way. The criticality factor is 1.0 for production systems and 0.5 for non-critical workloads. Available capacity is normalized to [0,1] representing the percentage of free resources. The proximity score is 1.0 for neighbors that are directly connected and decreases by 0.2 for every network hop.

V. IMPLEMENTATION IN IFOGSIM2

We selected iFogSim2 [3] because it is designed for fog computing, provides realistic modeling of network, and Java-based flexibility. Any fog cluster has 1 Gateway Node (8-core CPU at 3.0GHz, 16GB RAM), 1 Master Node (8-core CPU at 3.0GHz, 16GB RAM), 5-8 Fog Nodes (4-core CPU at 2.5GHz, 8GB RAM), and 50-100 IoT Devices. The full simulation topology consists of 4 fog clusters that have realistic geographic distances from one another.

We created models for three types of IoT workloads. The first is called Periodic Monitoring where sensors update every 10

to 60 seconds. The second model is called Event-Driven Traffic, where surges of activity occur when a security camera captures movement. The last one is Continuous Streaming, where industrial sensors send constant data. Load imbalances among these three workload types are represented by geographic hotspots, where one geographic region has 80% of the overall load, while other regions each have only 40%. Additionally, we modeled time-based spikes in traffic, which represent flash crowd scenarios, and cascading events. To extend iFogSim2 we developed a number of custom components including an Inter Cluster Gateway class that allows for communication between gateways; a Master Node Coordinator class which coordinates resource negotiation across multiple masters; and lastly, a Master of Masters Logger class that enables centralized logging of all inter-cluster activity within a time-series database that can be used to integrate with other systems.

Implementation Challenges and Solutions.

The implementation of the proposed system raised numerous concerns that needed to be resolved through detailed analysis and development. The first of these concerns is related to having consistent availability when fog nodes are migrating. Fog nodes typically have requests in the queue when they migrate from one cluster to another, and these requests need to be processed without disruptions. To that end, we've included a migration checkpoint protocol. This protocol enables fog nodes to process requests with less than 2 seconds remaining processing time before migrating to other areas. It also allows fog nodes to forward pending requests that may not be fully processed yet to other nearby fog nodes that can accommodate the outstanding requests. With this protocol in place, the percentage of dropped requests during migration reduced from 12% to 1%

The prediction of load in an accurate way is another challenge. It has a great influence on the effectiveness of mode selection. Instead of relying on the simple prediction style we built adaptive load balancing principles [2] by using a hybrid model for the prediction that unites exponential weighted moving average (EWMA) for tracking the short-term trends by giving attention to data and pattern matching for occasional workloads. The EWMA uses $\alpha=0.3$ and focuses on the recent data while having the historical context kept. In smart city deployments, where there is predictable routine every day, the pattern matcher was 89% exact when predicting load spikes 5-10 minutes in advance, allowing the activation of proactive sharing of resources.

The separation of the network was another problem experienced by clusters that practiced federated clustering. Although disconnected from the master of masters, a cluster continues to operate independently as before, but cannot create new arguments to share resources with other clusters. To address this, a mechanism was employed that utilizes a local cache of its neighbor's last-known state (load and reciprocity score) for greater than thirty minutes. In the event of a partition, the cluster can utilize the cached neighbor state to assist in formulating decisions on how to share resources. Once the partition is removed, requests are renegotiated and fulfillment can resume with 94% of the time required to share resources in the event of network failures lasting over 15 minutes.

VI. EXPERIMENTAL EVALUATION

The system was tested under four different scenarios: Geographic Hotspot (Cluster A is at 90% load, others are at 40-50%, 2 hours), Temporal Spike (sudden 95% spike for 15 minutes), Cascading Load (the first spike triggers secondary spikes), and Gradual Growth (increases from 50% to 85% over 4 hours). Each case lasted for 2-4 hours with 10 independent runs for every scenario.

To compare the performance three configurations were needed: Baseline (HFMCFA without inter-cluster sharing), Redirection-Only (request redirection is allowed), and Full System (allows both redirection and migration). In the first table request drop rates across different scenarios are shown.

TABLE I

REQUEST DROP RATES ACROSS SCENARIOS (%)

Scenario	Baseline	Redirection	Full System
Geographic Hotspot	12.4	4.2	0.8
Temporal Spike	8.7	2.9	0.3
Cascading Load	18.3	7.1	2.6
Average	**11.2**	**4.0**	**1.1**

The Full System cuts request dropping 90% when compared to baseline. Geographic hotspot case shows the most impressive improvement (from 12.4% to 0.8%). Node migration is an important factor because without it the system will have a heavy load. It is shown in cascading load case (18.3% to 2.6%).

In the second table comprehensive system-wide performance metrics are presented. The Full System has improved resource use for 42%, a decrease for 57% in utilization variance, availability improvements by 0.9 percentage points, energy saving for 14.6%, and 138% MTBF improvement.

TABLE II

SYSTEM-WIDE PERFORMANCE COMPARISON

Metric	Baseline	Redirection	Full System
Avg Utilization	58.3%	81.7%	+40.2%
Availability	98.3%	99.2%	+0.9pp
Energy Savings	847.3 kWh	723.6 kWh	-14.6%
MTBF (hours)	47	112	+138%

An increase for 42% in average resource use is an important factor. The 57% reduction in utilization variance shows that the load is more balanced. These improvements are maintained by the system while also growing general availability, which makes it clear that resource sharing enhances the system reliability in spite of putting it at risk. The master-of-masters logging system keeps latency at

6.2ms, reaches 11.4ms at 95th percentile, and does 8,400 transactions in a second, which means that it both hits and goes beyond its performance goals.

VII. DISCUSSION

The results of the test show that request redirection and node migration work together instead having competing strategies. Request redirection reacts fast to load spikes without requiring heavy coordination, while node migration allows long-lasting imbalances that would overwhelm the single cluster. An important safety mechanism is the 30% sharing limit, which keeps the donor cluster degradation under 1% at the same time stopping the cascade failures. If sharing is unlimited, donor clusters will be under a heavy load because of helping their neighbors and the cascade failure rate will be around 18.7%.

When we compare with traditional cloud load balancers, there are a lot of clear differences. Cloud load balancers work in homogeneous environments with many resources. Our federated fog approach must work with limitations of geographical areas and self-governing of the cluster, with heterogeneous capacity-limited resources, and keep the guarantees of local services while also dealing with neighbors. The limit of sharing, which is 30%, and negotiation rules solve these fog-specific problems.

A. Security and Trust Considerations

Resource sharing brings some problems of security that don't exist in isolated cluster systems. Malicious clusters could possibly lie to the system with incorrect reports about low load so they would attract the resources, but then they use borrowed nodes for unauthorized workloads. The mechanism of reciprocity offers the minimal protection by limiting access for clusters that only keep consuming without giving back. However, stronger attacks need extra protection. Three security layers were used to solve these issues. The first one is that mutual TLS 1.3 authentication allows only clusters that have the right to take part in sharing the resource. Second is that the master-of-masters finds out anomalies in transaction history, and flagging clusters which have reported load metrics that vary for a great deal from historical norms (z-score > 3.0). In testing, 97% of fake reporting attempts was spotted by this mechanism and it got wrong only 2.1%. Third is that migrated nodes keep encrypted channels of communication with their home clusters so they allow audit logs to verify that resources are used correctly.

Isolation of data is another aspect of security that can arise due to how requests to IoT resources are directed. Requests for sensitive IoT data from Cluster A to Cluster B will flow through the infrastructure of Cluster B. Applications with strict requirements that their data remain in specific locations - for example, a healthcare sensor that is subject to HIPAA regulations or a financial IoT device subject to PCI-DSS standards - could experience regulatory compliance issues if the data are transferred. We anticipate the problem can be managed through a policy-based approach that allows clusters to determine what types of data may be shared on resource sharing. For example, very sensitive data would remain within the geographic region of the originating cluster even at the cost of some additional requests being dropped; on the other hand, less sensitive data would benefit from inter-cluster load balancing.

When each cluster connects well to the network, the maximum performance of the nodes will be achieved. On the other hand, when the network is separated, the clusters are isolated, and there will be no resource sharing between them. Each cluster will continue operating autonomously, and as a result, the performance of that cluster will drop to baseline levels. By anticipating resource demand and planning for resources in advance, fewer instances of real-time communication between clusters will be needed.

B. Comparative Analysis with Alternative Approaches

A comparison was done between our system of inter-cluster resource sharing and three other architectural methods to confirm its advantages. The baseline HFMCFA system (without an inter-cluster sharing) forms the beginning. Cloud forwarding sends the requests straight to public cloud resources when local capacity runs out, taking advantage of cloud flexibility at the cost of risen latency. With static cluster federation 10% of resources are reserved to help the clusters without considering the actual conditions of load.

For the geographic hotspot model, cloud forwarding lowered request dropping to 3.2% vs. a dropped 0.8%, with a latency penalty of 142ms average instead of our 23ms for forwarded requests. The cloud approach becomes viable if the latency tolerance is above 100ms but falls short for real-time applications, such as industry control or autonomous driving, that require sub-50ms response. Static federation with 5.4% request dropping which is better than baseline but much worse than our dynamic scheme. The fixed 10% parameter was not enough for the peak load situations and wasted precious resources on normal traffic.

VIII. SMART CITY DEPLOYMENT

To provide an example of how the smart cities model can be applied in the real world, we performed an in-depth study of four Urban Districts - the Business District, Residential Zone, Industrial Park, and University Campus - each with its own fog cluster system that manages data differently based on that cloud's traffic and Quality of Service (QoS) needs. The Business District has the largest number of surveillance cameras (5,000), traffic sensors (2,000), and environmental monitors (500), and these devices only operate during the weekday business hours (8 AM to 6 PM Monday through Friday), generating a total daily IoT data load of 1.2TB, and must comply with very strict latency requirements of 50 ms for real-time traffic control. The Residential Zone is highest during the evening peak period, with an estimated daily data load of 0.800 TB produced by the use of 8,000 smart devices, 1,500 security cameras, and 300 utility meters, but has a much more relaxed latency requirement of 200 ms for real-time operation. Three shifts daily and more consistent upload patterns (all day long) for businesses create approximately 3000 sensors, 500 automated guided vehicles, 200 quality control cameras (total daily upload 2.1TB) in the Industrial Park, with ultra-low latency (10MS) for machine operations. The University Campus has a bimodal upload pattern driven by classroom hours (9AM-5PM) where there are approximately 10000 students using devices, and also driven

by late evening study periods (8PM-12AM). For the university, there are 800 sensors uploading 1.5 TB of data daily, with a moderate (100 MS) latency for IoT operations.

Without sharing fog resources between clusters, every district provides fog capacity only for peak loads. The breakdown of capacity utilization by cluster shows: the Business District averages 48% utilization across all hours (95% during peak, 20% during downtime); the Residential Zone 41% (90% evening, 15% daytime); the Industrial Park 87% (load remains the same consistently); and the University Campus 52% (85% occupied when classes are in session, 25% during downtime). Therefore, the system-wide average utilization of fog resources only reaches 57% due to the overall system's inability to absorb the extreme loading conditions of each individual cluster at any point in time.

Since implementing the inter-cluster resource sharing system, our operations have entirely changed. The Business District averages a 92% usage rate Monday to Friday during the afternoon hours and offers capacity support to the Residential Zone (38% usage). The total number of requests to both zones averages 400 and reduces the Business District's overall capacity to 78% while increasing the Residential Zone's capacity to 51%. During evening hours the relationship is reversed, with the Residential Zone having a 89% usage rate and receiving support from the Business District with a 35% capacity. Previously, the University Campus and Industrial Park were both exchanging capacity between them during the shift change times when the University Campus was experiencing a lunch rush (12:00 - 1:00 PM). This time-frame coincided with the Industrial Park's lunch change-over. Reciprocity scores have been stable between the ranges of 0.94 - 1.08 for all clusters and indicate that the overall exchange of support between clusters has remained even, regardless of the differing traffic patterns for each cluster.

System-wide metrics had major improvements: an increase in average utilization to 79% (+22 percentage points), decrease in requests being dropped from 8.4% to 1.1% (-87%), and 99.4% of latency-sensitive requests continued to meet QoS compliance. The city avoided $180,000 of planned fog infrastructure expansion and took full advantage of its current resources. With reducing the number of idle servers using standby power, energy consumption decreased by 16%. This case study illustrates that inter-cluster resource sharing creates tangible value in the use of heterogeneous workloads and complementary load patterns.

IX. CONCLUSION

This paper presents a new inter-cluster resource sharing scheme that is applicable to hierarchical federated fog computing. The scheme involves using request redirection to dynamically redistribute load among nodes within the clusters, thus allowing for quick response times. A second operation is to allow nodes to migrate temporarily to other nodes to provide continued service when needed. The intelligent coordination of master nodes in each cluster, with a maximum of 30% cluster-to-cluster resource sharing, provides an effective and practical way to achieve an elastic form of resource sharing while preserving the overall stability of the system.

The results reflect a 68% drop in request drop rates, an increase in resource usage of 42%, and an overall system availability percentage of 99.2%. The 30% cap is effective in preventing chain failures and large-scale cross-assist cases, while the Master-of-Masters logs provide less than 8 ms of latency and global visibility with Master-of-Masters log. The Master-of-Masters log events also provided a combination of 14.6% energy savings resulting from better load balancing across devices.

Research needs to evaluate how to apply machine-learning technologies to create more accurate predictions of future workloads based on current activity, allow multiple levels of resource sharing, enable users to dynamically allocate shared resources based on QoS requirements, and enhance the security of all data contained within these shared environments. As IoT continues to expand into more and more use cases, the ability to effectively share resources between federated fogs will become increasingly important

REFERENCES

[1] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in Proc. MCC Workshop Mobile Cloud Computing, 2012, pp. 13-16.

[2] N. Naltakyan, "Load Balancing in Adaptive Fog Computing: Research Problems and Solutions Framework," in Proc. CSIT Conf., Yerevan, Armenia, Sep. 2024, pp. 344-347.

[3] H. Gupta et al., "iFogSim: A toolkit for modeling fog computing," Software: Practice and Experience, vol. 47, no. 9, pp. 1275-1296, 2017.

[4] R. Mahmud, R. Kotagiri, and R. Buyya, "Fog computing: A taxonomy," in Internet of Everything, Springer, 2018, pp. 103-130.

[5] M. Mukherjee, L. Shu, and D. Wang, "Survey of fog computing," IEEE Commun. Surv. Tut., vol. 20, no. 3, pp. 1826-1857, 2018.

[6] T. Qayyum et al., "Mobility-aware hierarchical fog for IIoT," J. Cloud Comput., vol. 11, art. 64, 2022.

[7] A. Singh et al., "Server-storage virtualization," in Proc. SC'08, 2008, pp. 1-12.

[8] V. Cardellini et al., "Dynamic load balancing," IEEE Internet Comput., vol. 3, no. 3, pp. 28-39, 1999.

[9] R. Xu et al., "Workflow scheduling in cloud-fog," in Int. Conf. BPM, Springer, 2021, pp. 337-347.

[10] D. Puthal et al., "Secure load balancing in fog," IEEE Commun. Mag., vol. 56, no. 5, pp. 60-65, 2018.